

F3: Klasser

Malmö universitet, Institutionen för datavetenskap och medieteknik

Innehåll

- Objekt
- Skapa klasser
- Sammanfattning



Objekt

Innehåll

- Objekt sen tidigare
- Vad är ett objekt?
- Varför använda objekt?

Objekt sen tidigare

I Python är allt man kan spara objekt.

Alla `int` är objekt, alla `float`, och alla `str` .

Lite förenklat kan man säga att allt man kan sätta en punkt efter är objekt.

I tidigare kurser har vi dock inte tittat på dem på det viset.

I Java är `int`, `float`, och `double` så kallade *primitiver*, och mer basic än objekt

`String` är däremot objekt i Java

Saker vi kunde göra med str

Objekt har inbyggda *metoder* som är kommandon kopplade till objektet.

I Python kunde vi exempelvis skriva:

```
svar = input()
if svar.isdigit():
    print(svar, "innehåller bara siffror")
else:
    print(svar, "innehåller något annat än siffror")
```

Här är kommandot `.isdigit()` en metod kopplat till strängar.

Vad är ett objekt?

Ett objekt är en *instans* av en klass.

Vad är en klass?

En klass är en mall för att skapa objekt.

Ett exempel

Tänk dig en cykel. Begreppet cykel är en klass.

En specifik cykel är ett objekt. Till exempel min cykel.

Ett exempel

Alla cyklar har vissa egenskaper, exempelvis färg och antal växlar. Dessa kallas *attribut*.

Alla cyklar har också saker de kan göra, exempelvis rulla och bromsa. Dessa kallas *operationer*.

Varför använda objekt?

Objekt hjälper oss att strukturera och organisera vår kod.

Objekt kan användas för att modellera verkliga saker och koncept.

Varför använda objekt? Ett exempel

Tänk att vi ska skapa ett program för att hantera cyklar.

Vi hade då haft två alternativ:

1. Skapa variabler och funktioner för varje cykel
2. Skapa en klass `Cykel` och sedan skapa objekt av den klassen



Skapa klasser

Innehåll

- Skapa klasser
- Konstruktor
- Attribut
- Operationer
- Åtkomstidentifierare

Skapa klasser

- I Java behöver varje klass ligga i sin egen fil.
- Varje fil måste heta samma sak som klassen.

Skapa klasser

```
public class Bike {  
    private String colour ; // Attribut  
    private int gears ; // Attribut  
    public Bike (String colour, int gears){ // Konstruktor  
        this.colour = colour;  
        this.gears = gears;  
    }  
    public void roll(){ // Operation / metod  
        System.out.println("Cykeln rullar");  
    }  
    public void brake (){ // Operation / metod  
        System.out.println("Cykeln bromsar.");  
    }  
}
```

Skapa objekt

```
Bike myBike = new Bike("svart", 7);  
Bike yourBike = new Bike("röd", 21);  
  
myBike.roll();  
yourBike.brake();
```

I koden ovan *instansierar* vi två objekt av klassen **Bike**.

Sen använder vi objekten genom att anropa deras operationer/metoder

Konstruktor

```
public class Bike {  
    public Bike (String colour, int gears){  
        // Code goes here  
    }  
}
```

- En konstruktor är en metod som anropas när ett objekt skapas.
- Konstruktorn har samma namn som klassen och saknar returtyp.

Konstruktor

```
public Bike (String colour, int gears){ // Konstruktor
    this.colour = colour;
    this.gears = gears;
}
```

- I konstruktorn brukar vi vätta värden på objektets attribut.
- Nyckelordet **this** refererar till det aktuella objektet
- Om vi inte använder **this** skulle vi referera till parametrarna. Ha för vana att alltid använda **this** när du refererar till attribut.

Konstruktor: Flera konstruktorer

```
public class Bike {  
    private String colour ; // Attribut  
    private int gears ; // Attribut  
    public Bike(){ // Tom parameterlista  
        this.colour = "svart";  
        this.gears = 1;  
    }  
    public Bike (String colour, int gears){ // params  
        this.colour = colour;  
        this.gears = gears;  
    }  
}
```

Attribut

En variabel som är deklarerad i en klass kallas för ett *attribut*.

Attribut brukar deklareras överst i klassen.

```
public class Bike {  
    private String colour ; // Attribut  
    private int gears ; // Attribut  
  
    public Bike (String colour, int gears){  
        // Code goes here  
    }  
}
```

Attribut: Åtkomstmodifierare

Kommandona `private` och `public` kallas åtkomstmodifierare.

Attribut bör nästan alltid vara `private`.

Ett `private` attribut kan endas nå inifrån klassen.

Operationer

En metod som är deklarerad i en klass kallas för en *operation*

```
public class Bike{  
  
    public void roll(){ // Operation / metod  
        System.out.println("Cykeln rullar");  
    }  
  
    public void brake (){ // Operation / metod  
        System.out.println("Cykeln bromsar.");  
    }  
}
```


Operationer

```
Bike myBike = new Bike("svart", 7);  
myBike.roll();
```

Precis som att vi skriver `input.nextInt()` för att anropa metoden `nextInt` skriver vi `myBike.roll()` för att anropa metoden `roll`.

Operationer

I övrigt fungerar operationer som vanliga metoder (eller funktioner).

Åtkomstidentifierare

Ett attribut eller en operation med `public` åtkomstidentifierare kan nås från andra klasser.

Ett attribut eller en operation med `private` åtkomstidentifierare kan endast nås inom den egna klassen.

Åtkomstidentifierare: exempel

```
public class Bike {  
    public String colour ; // Publikt attribut  
    private int gears ; // Privat attribut  
    public Bike (String colour, int gears){  
        this.colour = colour;  
        this.gears = gears;  
    }  
}
```

Åtkomstidentifera

```
Bike myBike = new Bike("svart", 7);  
System.out.println(myBike.colour); // Fungerar  
System.out.println(myBike.gears); // Fungerar inte
```

Åtkomstidentifierare: get och set

En vanlig standard är att göra attribut `private` och sedan skapa `public` get- och set-metoder för att läsa och ändra värdet på attributen.

```
public String getColour(){  
    return this.colour;  
}  
  
public void setColour(String newColour){  
    this.colour = newColour;  
}
```



Sammanfattning

Sammanfattning

- Klasser är mallar för att skapa *objekt*
- Man kan skapa flera objekt av samma klass
- Objekt har *attribut* (variabler) och *operationer* (metoder)
- Attribut och operationer kan vara **public** eller **private**
- Konstruktorn är en särskil metod som anropas när ett objekt skapas
- **this** referear till det aktuella objektet

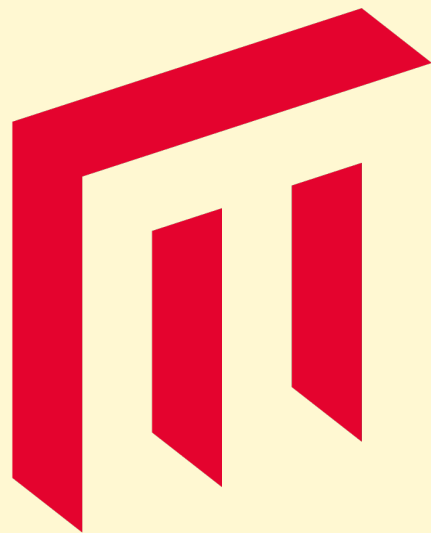
Sammanfattning

- Objekt hjälper oss att strukturera och organisera vår kod.
- Objekt kan användas för att modellera verkliga saker och koncept.
- Vi kommer att kolla på hur vi kan använda flera klasser i samma program senare.
- Det mesta som hjälper oss att läsa och strukturera vår kod är bra. Vi lägger mer tid på att titta på vår kod än att faktiskt skriva den.

Rekommenderad läsning

Dietel & Dietel, Kaptiel 7, 298–338

Tove Janson, *Det osynliga barnet*



**MALMÖ
UNIVERSITET**

